Strategic Solution — Customer Identity & Data Architecture

Fahrenheit Pharmacy — Sales Ledger

Final strategy draft · 28 June 2026 · Stack: React/Vite SPA on Supabase Postgres

Recommendation in one paragraph. Evolve the app on Postgres/Supabase (no re-platform, no microservices) toward a normalized domain model with a first-class `customers` entity, where the **mobile number is the primary customer identity** — normalized, partially-unique, **with an explicit override for shared family numbers and a first-class path for no-mobile walk-ins**. Move all credit/aggregation logic **into the database**, and roll it out as a phased strangler migration where each phase ships value and nothing breaks. The name becomes an editable *label* on a phone-keyed customer, which structurally eliminates the duplicate-identity bug class.

1. Root cause

Identity is free-text `customer_name` + `mobile_number` repeated on every `sales` line — no customer entity, no server-side aggregation. Every issue found (Dashboard ≠ Reports credit, double-subtraction risk, autocomplete misses, duplicate customers, the RAHUL/RAHUL-PANDEY and NEERAJ/SHARAD cases) traces to those two gaps.

2. The identity model (locked decision)

Mobile number is the primary identity — "one number = one person *by default*, with overrides."

RULE	DECISION
Primary key for a customer	Normalized mobile (`phone_digits` = last 10 digits)
Name	An editable **label** on the phone-keyed customer — never the identity
Two real people on one phone (families)	**Allowed via override** — extra customers carry `shares_phone = true`
Uniqueness enforcement	**Partial** unique index: one non-shared customer per number; blanks excluded
No-mobile / walk-in sales	**First-class path** — `customer_id` nullable; anonymous or name-only
Typos	Confirm step shows name prominently; **edit-name** fixes labels without forking
Junk numbers	Light validation (reject all-same-digit like `0000000000`)
Search	Mobile-first (primary) **and** name autocomplete (secondary) — both kept

```
-- one non-shared customer per number; blanks & deliberate family members excluded
create unique index uq_customer_phone on public.customers (phone_digits)
  where phone_digits <> '' and not shares_phone;
```

3. Target data model

```
organizations (id, name)                      -- tenant root (SaaS-ready, add early)
branches      (id, org_id, name)
app_users     (id, org_id, auth_uid, role)    -- owner / pharmacist / cashier

customers (id, org_id, name, phone,
           phone_digits  GENERATED last-10,    -- identity / lookup / uniqueness
           name_norm, phone_norm GENERATED,    -- keeps credit view provably equivalent
           shares_phone bool, created_at, updated_at)

invoices  (id, org_id, branch_id, invoice_no UNIQUE per org,
           customer_id FK, sale_date, payment_mode, bill_discount,
           total, status, created_by)
sale_items(id, invoice_id FK, medicine_name→product_id later,
           qty, rate, line_total, ...)

products      (id, org_id, name, ...)          -- medicine master (later)
stock_batches (id, product_id, batch_no, expiry, mrp, qty_on_hand)
```

```
payments  (id, org_id, customer_id FK, invoice_id NULLABLE, amount, mode, paid_date)
audit_log (id, org_id, entity, entity_id, action, diff, user_id, at)
```

DBA essentials baked in: exact `numeric` money; **FK indexes** (Postgres won't auto-create); `updated_at` triggers; invoice `status` for returns/voids; `pg_trgm` /normalized search in the DB; RLS keyed to `org_id` ; materialized views + `pg_cron` for heavy reports; partition `sale_items` by month only past a few million rows.

4. Phased roadmap (strangler — each phase ships, nothing breaks)

PHASE	DELIVERABLE	STATUS
0	`customer_outstanding` view; both screens read it (JS fallback)	BUILT
1	`customers` + `phone_digits` + backfill + `customer_id` FK; repoint view; pay-off writes `customer_id`	DRAFTED
1.5	Reviewed data cleanup (merge duplicates, flag families) → **then** add partial-unique phone index	DRAFTED
2	Mobile-first Sale Entry UX + edit-name + override prompt	TO BUILD
3	`invoices` header + `sale_items` split (column map done)	TO BUILD
4	`org_id`/branches + RLS-by-tenant + roles	TO BUILD
5	`products` + `stock_batches` (inventory), integrations	TO BUILD

Cross-cutting, start now: Supabase CLI migrations (versioned SQL), a staging database, generated DB types, and confirm PITR backups.

5. Sale Entry flow (Phase 2)

```
[ Mobile # ] ← primary, cursor starts here
   │ on 10 digits → lookup phone_digits
   ├─ 1 match   → prefill name → shopkeeper CONFIRMS (name shown prominently) → bill
   ├─ several   → pick family member, or "add another person on this number" (shares_phone)
   └─ none      → ask name → save (number tagged to name)
[ No number ]  → explicit walk-in path (customer_id null)
[ Edit name ]  → correct a typo'd label without forking the customer
```

Name autocomplete remains as the secondary lookup.

6. Credit handling (correctness — solved & wired)

- `customer_outstanding` view is the **single source of truth**; both screens read it.
- After Phase 1 it groups by `customer_id`; pay-offs write `customer_id`; the `paid` CTE **falls back** to name+mobile so the transition can't drop a payment.
- **Verified** identical to current logic — **₹22,365**, 0 orphans, money preserved.

7. Data cleanup

Three tiers, run after Phase 1, as `customer` merges via a `merge_customer()` function with an **auto-abort guard if total outstanding changes**:

TIER	CASES	ACTION
1 — safe	8 blank-mobile backfills (same name, one real number)	Active
2 — confirm	RAHUL→RAHUL PANDEY; GYANENDRA canonical	Owner decision
3 — judgment	SHARAD's stray number; TRIPATHI / VIVEK PATHAK	Owner decision

8. Sequencing & verification gates (the one rule that matters)

Phase 1 ships as a unit (migration + pay-off `customer_id` + view fallback). Then: **clean the data → add the partial-unique phone index → build the mobile-first UX.** You cannot enforce phone-uniqueness before the family/duplicate rows are resolved.

GATE	EXPECTED
Phase 1 applied	~224 customers · 0 orphaned rows · outstanding ₹22,365
Cleanup run	Total unchanged · customer count drops by # merges

GATE	EXPECTED
Mobile-first UX	Test pay-off reduces balance and persists after refresh

9. Guardrails

DO

- Evolve incrementally
- Phone-primary *with* override + no-mobile path
- Aggregation in SQL
- Tenancy column early
- Version every migration; backup first; staging first

DON'T

- Rewrite from scratch / microservices
- Leave Postgres
- **Hard** unique-mobile (breaks families)
- Enforce uniqueness before cleanup
- Pre-partition / over-index early

10. Status & immediate next steps

Built & verified (uncommitted): credit consistency (Phase 0), autocomplete fix + soft warning, pay-off `customer_id` plumbing.
Drafted: Phase 1 migration; cleanup SQL (needs Tier 2/3 decisions).
To build: `phone_digits` + finalize unique index; mobile-first Sale Entry + edit-name; Phases 3–5.

1. **You:** adopt Supabase CLI migrations + a staging DB; confirm PITR.
2. **Dev:** add `phone_digits` to Phase 1; write `finalize_phone_identity.sql` (partial-unique index, after cleanup).
3. **You:** dry-run Phase 1 → cleanup → finalize on staging; verify gates.
4. **Dev:** build mobile-first Sale Entry + edit-name (Phase 2).
5. Commit the already-built app fixes in focused commits.